POSSUMIC
正和微芯

RS6x_7x HIF Development

Quick Start Guide

Number of Pages:26

Document Version:1.1

Release Date:2026-08-14

Revision History

Version	Revision Description	Date
1.1	Modified the formatting.	2026-08-14
1.0	Initial release.	2026-06-30

Table of Contents

Revision History ii
Table of Contents iii
List of Figures iv
List of Tables v
1 Introduction 6
1.1 Features 6
1.2 Development Procedure 7
2 Development-Board Evaluation 8
3 Development Evaluation 9
3.1 Key Evaluation Items for Each Platform 10
3.1.1 RTOS Platform 11
3.1.2 Linux Platform 11
3.2 Resource and Timing Evaluation 11
3.2.1 Timing Evaluation 11
3.2.1.1 Point Cloud Evaluation 11
3.2.1.2 Data Cube Evaluation 12
3.2.1.3 Other Data Evaluation 13
3.2.2 Memory Resource Evaluation 14
3.2.2.1 Device Memory 14
3.2.2.2 Host Memory 15
3.2.3 I/O Resources 15
4 Development and Debugging 17
4.1 Basic-Function Adaptation 17
4.1.1 Host Porting 17
4.1.2 Porting and Debugging 17
4.1.2.1 I/O Notification Flow 19
4.1.2.2 Data Transmission Flow 19
4.1.2.3 Data Reception Flow 19
4.2 Advanced-Function Adaptation 20
4.2.1 SPI Transmission Performance Test 20
4.2.2 Burn Debugging 21
4.2.3 Multiple-Slave Debugging 21
4.2.4 Sample Debugging 21
5 Appendix A: HIF Radar Reporting Timing 22
5.1 Point-Cloud Data Reporting 22
5.1.1 Single-Frame Processing 22
5.1.2 Multi-Frame Processing 22
5.2 Data Cube Reporting 23
5.2.1 Single-Frame Reporting and Processing 23
5.2.2 Single-Frame Reporting and Multi-Frame Processing 23
5.2.3 Multi-Frame Reporting and Processing 24
5.2.4 Other Information Reporting 24
6 Appendix B: Host Real-Time Parameters 25

List of Figures

Figure 1-1 HIF Development Procedure 7
Figure 3-1 Point-Cloud Transmission Parameter Configuration 11
Figure 3-2 Data Cube Transmission Parameter Configuration 12
Figure 3-3 Other Data Transmission Parameter Configuration 14
Figure 3-6 I/O Resources 16
Figure 4-1 Development and Debugging Procedure 17
Figure 4-2 Port Sample UART Transmission Flow 18
Figure 4-3 Port Sample SPI or I^2C Transmission Flow 18
Figure 4-4 I/O Notification Flow 19
Figure 4-5 Data Transmission Notification Flow 19
Figure 4-6 Data Reception Notification Flow 20
Figure 4-7 Host Real-Time Parameters 20
Figure 5-1 Radar Frame Period 22
Figure 5-2 Single-Frame Point-Cloud Reporting 22
Figure 5-3 Multi-Frame Point-Cloud Reporting 23
Figure 5-4 Single-Frame Data Cube Reporting and Processing 23
Figure 5-5 Single-Frame Data Cube Reporting and Multi-Frame Processing 24
Figure 5-6 Multi-Frame Data Cube Reporting and Processing 24
Figure 6-1 Host Real-Time Parameters 25

List of Tables

Table 1-1 HIF Features 6
Table 2-1 Basic Samples 8
Table 3-1 Reference Data for Typical Applications 9
Table 6-1 Host Real-Time Parameters 25

1 Introduction

HIF stands for Host Interface and is the data transmission protocol between a controller and a Device. When the radar chip operates as a peripheral (Device), HIF is used for communication between the controller (Host) and the radar chip, including Command configuration, data reporting, and firmware burn.

This document guides software engineers through the rapid use of HIF in a solution. For details about HIF, see the'*RS6x_7x_HIF_Reference Manual*'.

1.1 Features

HIF supports two specification versions. Normal is used by default. The features are listed in the following table:

Table 1-1 HIF Features

Item			Mini	Normal
Scenario	Application Scenario		• Algorithms and applications are developed on the Device, without high-volume data reporting • Low-power application development	• Algorithms and applications are developed on the Device, without high-volume data reporting • High-volume data reporting (1D/2D/ADC/high-density point cloud) • Application algorithms are developed on the Host
	Data Type		• Point-cloud reporting • Application-result reporting • Application or debug-information reporting	• Data Cube reporting • Point-cloud reporting • Application-result reporting • Application or debug-information reporting
Basic	Communication	UART	• 115200 • 460800 • 921600 • 1000000 • 2000000	• 115200 • 460800 • 921600 • 1000000 • 2000000
		SPI	• Std-SPI • <= 8 MHz	• Std-SPI • Dual-SPI • Quad-SPI • <= 64 MHz
		I^2C	• 100 kHz • 400 kHz	• 100 kHz • 400 kHz
	Basic Functions	Concurrency	Supported	Supported
		Blocking reporting	Supported	Supported
		Non-blocking reporting	Supported	Supported

Item			Mini	Normal
		Message validation	Supported	Supported
		Extended header	Not supported	Supported (not supported in the current version)
		PM	Supported	Supported
		Reporting through DMA	Not supported	Supported
		Zero-copy	Supported	Supported
	Storage Resources	Thread Stack (Bytes)	512	1024
		Reporting queue	- Frame Node: 4 - Data Node: 16 - Semaphore Node: 2	- Frame Node: 16 - Data Node: 64 - Semaphore Node: 4
		Command and Response buffer (Bytes)	256	512
Extended	Burn		Supported	Supported
	OTA		Not supported	Supported (not supported in the current version)
	Multiple slaves		Supported	Supported
	Application-layer fragmentation		Not supported	Supported (disabled by default)
	Application-layer retransmission		Not supported	Supported (disabled by default)
	Transport-layer fragmentation		Not supported	Supported
	Transport-layer retransmission		Not supported	Supported (disabled by default)
	Encryption and decryption		Not supported	Supported (disabled by default)
	XIP/SRAM execution		SRAM (XIP optional)	SRAM (XIP optional)

1.2 Development Procedure

PSIC radar products support two development modes: development-free and SDK development. To use HIF in a radar application, follow the procedure below:

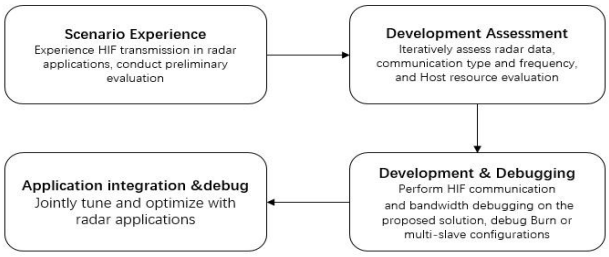


Figure 1-1 HIF Development Procedure

2 Development-Board Evaluation

Possumic Technology provides evaluation Samples for different types of radar application data (motion point clouds, micro-motion point clouds, 1D Data Cubes, and 2D Data Cubes). Users can use a development board and PC Tools to evaluate radar applications and HIF transmission.

This process helps users achieve the following objectives:

1. Perform a preliminary evaluation of HIF communication

   Different Samples have different HIF transmission-bandwidth requirements and can be used as references when evaluating the communication type and rate. For example, when transmitting 1D Data Cube data, HIF can use SPI or UART, but not I^2C.

   Based on a Sample, users can also use the Tool to adjust radar parameters and the HIF communication type and frequency. Checksum errors, dropped frames, and other prompts can then be observed immediately to determine whether the transmission bandwidth is sufficient.

2. Understand the HIF communication protocol for reference during porting and debugging

Use Tool logs together with the documentation to understand the HIF Frame format and interaction flow. During subsequent development and porting, use these logs for Debug comparison.

Table 2-1 Basic Samples

Sample Name	Report Data Type	Default Parameter	Communication
Tracking	Motion point cloud	Frame period: 50 ms Motion points: max. 9 KB/frame	SPI: Standard, 60 MHz (default) UART: 1000000 bps (optional) I^2C: 400 kHz (optional; number of radar point-cloud points < 128)
Posture	Motion point cloud, Micro-motion point cloud	Frame period: 50 ms Motion point cloud: max. 9 KB/frame Micro-motion point cloud: max. 3 KB/frame	SPI: Standard, 60 MHz (default) UART: 1000000 bps (optional)
Sleep	Motion point cloud, Micro-motion point cloud, Interval0 1D Data Cube	Frame period: 50 ms 1D Data Cube: 8 KB/frame Motion point cloud: max. 9 KB/frame Micro-motion point cloud: max. 3 KB/frame	SPI: Standard, 60 MHz (default) UART: 1000000 bps (optional)
ReportDataCube1D	1D Data Cube	Frame period: 5 ms 1D Data Cube: 64 KB/8 frames	SPI: Standard, 64 MHz (default) UART: 1000000 bps (optional)
ReportDataCube1D	2D Data Cube	Frame period: 100 ms 2D Data Cube: 256 KB/frame	SPI: Standard/Dual/Quad, 64 MHz (default)

For details about the Samples, see the *'RS6x_7x_mmWave_Radar_Sample_Reference_Manual'*.

3 Development Evaluation

During the Scenario Evaluation phase, users can use RadarDebugTool to evaluate the radar and preliminarily determine the radar parameters and HIF communication type. This chapter describes the parameters that require particular attention during evaluation and explains how to balance and iteratively adjust the communication type and rate, transmission strategy, and radar parameters to obtain the optimal solution.

Algorithms are central to radar development. Based on where the radar algorithm runs, the system can be classified as either Device-side algorithm processing or Host-side algorithm processing. When the algorithm runs on the radar Device, the data reported to the Host is typically the processed algorithm result. The reported data volume is small, and communication has almost no impact on algorithm development, as in the occupancy-sensing application shown below. When the algorithm runs on the Host, a large volume of data is reported and communication occupies a significant portion of each frame period. In this case, HIF communication is an important radar-design constraint. The following sections focus on Host-side algorithm processing.

The following table lists key parameter data for typical applications as a design reference.

Table 3-1 Reference Data for Typical Applications

Radar Solution	Data Type	Radar Param	Communication		Host		
			Types	Trans Time (ms)	CPU Max Calc Time (ms)	Max response latency (ms)	SramSize (KB)
Ultra-Low-Power Human Presence	Presence Detection	- Report state - Period (ms): 1000 - TxDuration (ms): 0.2	UART 115200	0.1	1000	>= 1000	0
			I2C 100KHz	0.1	1000	>= 1000	0
Tracking	Point Cloud	- Motion Cloud: 256 - Present Cloud: 128 - Period (ms): 50 - TxDuration (ms): 16.5	UART 2MHz	23.22	26.78	>= 1000	9
			SPI-Std 64MHz	1.21	48.79	>= 1000	9
	Point Cloud	- Motion Cloud: 512 - Present Cloud: 256 - Period (ms): 50 - TxDuration (ms): 16.5	SPI-Std 64MHz	2.24	47.76	>= 1000	18
	Data Cube	- Data Cube (KB): 256 - mmw Period (ms): 100 - TxDuration (ms): 16.5	SPI-Std 64MHz	43.26	56.74	40.24	256
			SPI-Dual 64MHz	26.78	73.22	56.72	256
	Data Cube	- Data Cube (KB): 512 - mmw Period (ms): 100 - TxDuration (ms): 16.5	SPI-Dual 64MHz	52.89	47.11	30.61	512
			SPI-Quad 64MHz	36.42	63.58	47.08	512
Posture	Point Cloud	- Motion Cloud: 768 - Present Cloud: 256 - Period (ms): 50 - TxDuration (ms): 16.5	SPI-Std 64MHz	2.78	47.22	47.22	24
	Point Cloud	- Data Cube (KB): 256 - mmw Period (ms): 50 - TxDuration (ms): 16.5	SPI-Dual 64MHz	26.78	49.42	6.72	512
			SPI-Quad 64MHz	18.59	49.37	14.91	512
	Data Cube	- Data Cube (KB): 512 - mmw Period (ms): 50 - TxDuration (ms): 16.5	SPI-Quad 64MHz	36.42	-2.92	-2.92	1024
	Point Cloud	- Motion Cloud: 768 - Present Cloud: 256 - Period (ms): 75 - TxDuration (ms): 36	SPI-Std 64MHz	2.78	72.22	72.22	24
Vital Monitoring	Data Cube	- Data Cube (KB): 1 - mmw Period (ms): 10 - TxDuration (ms): 0.2	UART 2MHz	5.23	4.77	>= 1000	2
	Cube	- Data Cube (KB): 8 - mmw Period (ms): 10 - TxDuration (ms): 0.2	SPI-Std 64MHz	1.56	8.44	158.44	16
Sleep Monitoring	Data Cube	- Data Cube (KB): 64 - mmw Period (ms): 20 - TxDuration (ms): 13.68	SPI-Std 64MHz	11.4	16.84	28.60	128
	Point Cloud + Cube	- Motion Cloud: 256 - Data Cube (KB): 8 - mmw Period (ms): 20 - TxDuration (ms): 13.68	SPI-Std 64MHz	2.72	17.28	>= 1000	22

As shown in the preceding table, each key parameter affects performance differently. To better understand the transmission model, see ***'Appendix A: HIF Radar Reporting Timing'*** *and* ***'Appendix B: Host Real-Time Parameters'***. The effects of the parameters are summarized below:

1. Max Calc time (available Host calculation time)

   The Host receives radar data from the Device for calculation (algorithm processing). When the application scenario and Host platform have been determined, the available calculation time is the key factor in determining whether the total computing capability meets the application requirements.

   Using the radar frame period as the reference interval, when the Host uses a single buffer, the calculation time equals the frame period minus the transmission time. When the Host uses multiple buffers, the calculation time spans the entire frame period, but the calculation capacity available during transmission must be derated using the Host CPU utilization rate input parameter. The result depends on the Host strategy. For example, when a dedicated core in a multi-core system receives data, the entire interval can be used for calculation. When a single-core system uses DMA reception, most of the interval can be used for calculation. When a single-core system uses interrupt-driven or polling reception, essentially none of the transmission interval is available for calculation. In addition, a multitasking Host must reserve calculation time for other urgent tasks without affecting calculation of the current-frame data.

2. Max response latency (Host response latency)

   On a Host platform such as Linux, a non-real-time OS scheduling policy can prevent the Host from promptly responding to a radar-data read request or can schedule the task out while data is being read. For example, the interval from assertion of the Device notification I/O until the Host begins data transmission is typically tens of milliseconds. In high-frame-rate radar applications, if multi-frame transmission buffering is not available, data from the next frame may overwrite the current radar data before transmission is complete. Therefore, response latency is an important evaluation parameter for Linux-class Hosts and affects both the transmission strategy and transmission buffer.

3. Host Cache frames (Host data buffer depth)

   A Host with non-uniform computing availability, where calculation tasks are frequently preempted by higher-priority tasks, can buffer multiple frames. When the Host has high response latency, the Device must support buffering and transmitting multiple frames, and the Host must allocate corresponding buffers for reception and calculation. For high-frame-rate radar applications with small data volumes, the Device can also buffer several frames for transmission together, while the Host allocates corresponding buffers for processing.

4. Common Param (communication type and frequency)

   This parameter directly affects solution feasibility and is typically evaluated in terms of transmission time. It determines whether multi-frame buffered transmission is required and affects the sizes of the transmission and calculation buffers required by the Host and Device. During evaluation, first select a communication option based on the data type using the table above, and then fine-tune the parameters. The communication interfaces, supported frequencies, and I/O resources available on the Host are also decisive factors when selecting the communication method.

During evaluation, use the ***'Radar_HIF_Calculation'*** spreadsheet to fine-tune the parameters. The main steps are as follows:

1. Determine the radar parameters and reported-data type.

2. Use the Point Cloud, Data Cube, or Other Data sheet in *'Radar_HIF_Calculation'* to perform the evaluation calculation and obtain the key parameter data.

3. Use the Memory and I/O sheets in *'Radar_HIF_Calculation'* to calculate the required hardware resources, and compare the results with the actual Host resources to determine feasibility.

4. Repeat Steps 1 through 3 to obtain the optimal solution.

3.1 Key Evaluation Items for Each Platform

RTOS Hosts and Linux Hosts have significantly different characteristics and therefore require different resource-evaluation priorities.

3.1.1 RTOS Platform

RTOS Host characteristics:

1. Fast response
2. Limited SRAM resources (typically runs on an MCU)
3. Lower computing capability

Parameters requiring particular attention:

1. Host CPU Max Calc time (time available to the Host for calculation)
2. Host SRAM resources

3.1.2 Linux Platform

Linux Host characteristics:

1. Slow response
2. Abundant SRAM resources
3. High computing capability
4. Computing availability may be non-uniform (the CPU cannot be continuously dedicated to radar calculation and may be scheduled out to process other tasks)

Parameters requiring particular attention:

1. Host Max response latency
2. Uniform computing capability, or use multi-frame buffering on the Host

3.2 Resource and Timing Evaluation

3.2.1 Timing Evaluation

The following sections describe timing-related evaluations, including transmission time, available Host calculation time, and Host response latency, for different data types.

3.2.1.1 Point Cloud Evaluation

The point-cloud evaluation interface is shown below. Blue cells are input parameters and orange cells are output parameters.

Point Cloud

Input
Output
emp Output

INPUT

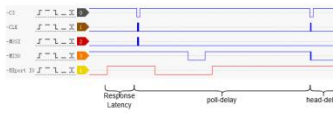
Radar Param	Frame Period	50	ms
	Start Freq	58	GHz
	Speed Resolution	0.165	m/s
	Range Resolution	0.04	m
	Max Speed	0.66	m/s
	Max Distance	10.24	m
	Antenna Mode	2T4R	
	Frame Structure	2DFFT	

Comm Param	UART	2000000	bps
	Stand SPI	64000000	Hz
	Dual SPI	64000000	Hz
	Quad SPI	64000000	Hz
	I2C	100000	Hz

Host Real-Time Param	Response Latency	70	us
	Poll Delay	550	us
	Packet Head Delay	47	us
	Inter-Packet Delay	106	us

Other Param	Motion Point Clout	128	
	Presence Point Cloud	128	
	Other time	0	ms
	Device Cache Frames	1	
	Host Cache Frames	2	
	Host CPU Utilization Rate	80	%

Radar Config	Radar TxDuration	13.8	ms
	Point Cloud Calc Time	9.7	ms



Proportion of CPU involvement computation during host trar

OUTPUT

Types	Device Param (Fixed trans data size)		Device Param (Fixed trans bandwidth)		Host param		
	Trans Time (ms)	Device Idle time (ms)	Max trans data(Bytes)	Max trasn point cloud	Trans Idle time	Max response latency	Max Calc time
UART	15.49	11.00	9770	814	34.51	34.51	46.90
Stand SPI	1.16	25.33	386015	32168	48.84	48.84	49.77
Dual SPI	0.97	25.52	772049	64337	49.03	49.03	49.81
Quad SPI	0.87	25.62	1544119	128677	49.13	49.13	49.83
I2C	282	-255.30	511	42	-231.78	-231.78	-6.36

Figure 3-1 Point-Cloud Transmission Parameter Configuration

Input parameters:

1. Radar Param (radar parameters): Enter values according to the radar configuration, for example, values obtained from RadarDebugTool.

2. Comm Param (communication parameters): Enter values according to the communication interfaces and frequencies supported by the Host. Multiple communication methods can be evaluated simultaneously.

3. Host Real-Time Param (Host real-time parameters): If exact data is unavailable, enter values based on Appendix A, or measure them using the port_device_time_meas project (for details, see Section 4.2.1, SPI Transmission Performance Test).

4. Other Param (other parameters):

+ Other Time: A reserved-time input. For example, reserve 5 ms per frame for processing high-priority tasks.

+ Device Cache Frames: Number of data frames that can be buffered on the Device for reporting. A value of 1 means that the current-frame data must be completely reported; a value of 2 means that two frames can be buffered for reporting. This parameter affects Max response time.

+ Host Cache Frames: Number of data frames that the Host can buffer for processing. This parameter addresses non-uniform Host computing availability.

+ Host CPU utilization rate: Proportion of the CPU available for radar calculation during Host transmission. This value depends on the Host reception strategy.

Output data:

1. Trans Time (transmission time): Evaluates whether sufficient time is available within the frame period to complete data transmission.

A positive margin indicates that sufficient time is available. A larger value indicates more margin and may allow the transmission rate to be reduced.

A negative margin indicates that the transmission time exceeds the frame period. Increase the transmission rate and check whether the margin becomes positive; otherwise, the communication method cannot be used.

2. Device Idle Time (radar Device idle time): Frame period minus the hardware transmit/receive interval and data-transmission time. For low-power applications, this value is used to evaluate the available sleep time.

3. Max trans data and Max trans point cloud: Evaluate the radar-data volume and parameters that can be supported at the highest transmission bandwidth supported by the Host.

4. Trans Idle time (Host transmission idle time): Frame period minus transmission time. This value is used to evaluate the time during which the Host can be fully dedicated to radar calculation.

5. Max response latency: Time from assertion of the Device I/O until the Host reads the data. This item is affected by Device Cache Frames, that is, the Device buffer size.

6. Max Calc time (time available to the Host for calculation): Used together with the average Host computing capability to evaluate whether sufficient calculation time is available for the current application.

3.2.1.2 Data Cube Evaluation

The Data Cube evaluation interface is shown below. Blue cells are input parameters and orange cells are output parameters.

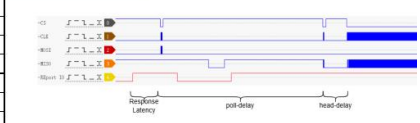
Data Cube

Input
Output
Temp Output

INPUT

Radar Param	Frame Period	20	ms
	Start Freq	58	GHz
	Speed Resolution	0.165	m/s
	Range Resolution	0.04	m
	Max Speed	0.66	m/s
	Max Distance	10.24	m
	Antenna Mode	2T4R	
	Frame Structure	2DFFT	

Host Real-Time Param	Response Latency	70	us
	Poll Delay	550	us
	Packet Head Delay	41	us
	Inter-Packet Delay	106	us
Other Param	Other time	0	ms
	Device Cache Frames	1	
	Host Cache Frames	2	
	Host CPU Utilization Rate	80	%



Proportion of CPU involvement computation during host transmission

Comm Param	UART	2000000	bps
	Stand SPI	64000000	Hz
	Dual SPI	64000000	Hz
	Quad SPI	64000000	Hz
	I2C	100000	Hz

Radar Config	Range FFT	256	
	Doppler FFT	8	
	Data Cube size	64	KB
	Radar TxDuration	13.8	ms

OUTPUT

Types	Device Param (Fixed trans data size)		Device Param (Fixed trans bandwidth)		Host param		
	Trans Time (ms)	Device Idle time (ms)	Max trans data(Bytes)	Max trasn data cube	Trans Idle time	Max response latency	Max Calc time
UART	329.66	-323.46	1230	1	-309.66	-309.66	-45.93
Stand SPI	11.51	-5.31	44384	43	8.49	8.49	17.70
Dual SPI	7.39	-1.19	88760	87	12.61	12.61	18.52
Quad SPI	5.33	0.87	177534	173	14.67	14.67	18.93
I2C	-	-	-	-	-	-	-

Figure 3-2 Data Cube Transmission Parameter Configuration

Input parameters:

1. Radar Param (radar parameters): Enter values according to the radar configuration, for example, values obtained from RadarDebugTool.

2. Comm Param (communication parameters): Enter values according to the communication interfaces and frequencies supported by the Host. Multiple communication methods can be evaluated simultaneously.

3. Host Real-Time Param (Host real-time parameters): If exact data is unavailable, enter values based on Appendix A, or measure them using the port_device_time_meas project (for details, see Section 4.2.1, SPI Transmission Performance Test).

4. Other Param (other parameters):

   + Other Time: A reserved-time input. For example, reserve 5 ms per frame for processing high-priority tasks.

   + Device Cache Frames: Number of data frames that can be buffered on the Device for reporting. A value of 1 means that the current-frame data must be completely reported; a value of 2 means that two frames can be buffered for reporting. This parameter affects Max response time.

   + Host Cache Frames: Number of data frames that the Host can buffer for processing. This parameter addresses non-uniform Host computing availability.

   + Host CPU utilization rate: Proportion of the CPU available for radar calculation during Host transmission. This value depends on the Host reception strategy.

Output data:

1. Trans Time (transmission time): Evaluates whether sufficient time is available within the frame period to complete data transmission.

   A positive margin indicates that sufficient time is available. A larger value indicates more margin and may allow the transmission rate to be reduced.

   A negative margin indicates that the transmission time exceeds the frame period. Increase the transmission rate and check whether the margin becomes positive; otherwise, the communication method cannot be used.

2. Device Idle Time (radar Device idle time): Frame period minus the hardware transmit/receive interval and data-transmission time. For low-power applications, this value is used to evaluate the available sleep time.

3. Max trans data and Max trans data cube: Evaluate the radar-data volume and parameters that can be supported at the highest transmission bandwidth supported by the Host.

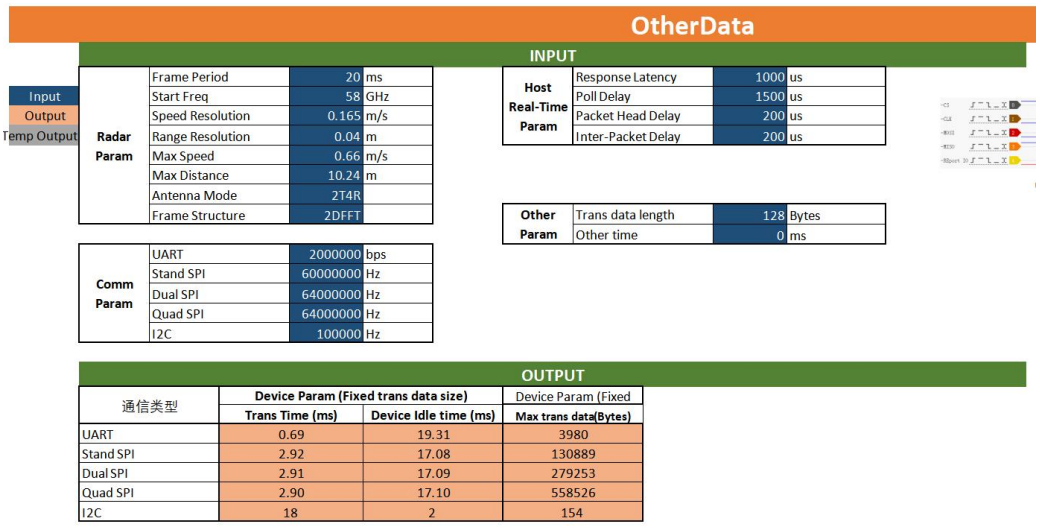
4. Trans Idle time (Host transmission idle time): Frame period minus transmission time. This value is used to evaluate the time during which the Host can be fully dedicated to radar calculation.

5. Max response latency: Time from assertion of the Device I/O until the Host reads the data. This item is affected by Device Cache Frames, that is, the Device buffer size. With multi-frame buffering, a larger number of buffered frames allows a longer delay. With single-frame transmission, the value equals the frame period minus the hardware-transmission time and data-transmission time. If the system response time exceeds this interval, data is overwritten.

6. Max Calc time (time available to the Host for calculation): Used together with the average Host computing capability to evaluate whether sufficient calculation time is available for the current application.

3.2.1.3 Other Data Evaluation

The evaluation interface for other data transmission, such as debug information or calculation results, is shown below. Blue cells are input parameters and orange cells are output parameters.



OtherData

INPUT

Input			
Output			
Temp Output			
Radar Param	Frame Period	20	ms
	Start Freq	58	GHz
	Speed Resolution	0.165	m/s
	Range Resolution	0.04	m
	Max Speed	0.66	m/s
	Max Distance	10.24	m
	Antenna Mode	2T4R	
	Frame Structure	2DFFT	

Host Real-Time Param	Response Latency	1000	us
	Poll Delay	1500	us
	Packet Head Delay	200	us
	Inter-Packet Delay	200	us

Other Param	Trans data length	128	Bytes
	Other time	0	ms

Comm Param	UART	2000000	bps
	Stand SPI	60000000	Hz
	Dual SPI	64000000	Hz
	Quad SPI	64000000	Hz
	I2C	100000	Hz

OUTPUT

通信类型	Device Param (Fixed trans data size)		Device Param (Fixed
	Trans Time (ms)	Device Idle time (ms)	Max trans data(Bytes)
UART	0.69	19.31	3980
Stand SPI	2.92	17.08	130889
Dual SPI	2.91	17.09	279253
Quad SPI	2.90	17.10	558526
I2C	18	2	154

Figure 3-3 Other Data Transmission Parameter Configuration

Input parameters:

1. Radar Param (radar parameters): Enter values according to the radar configuration, for example, values obtained from RadarDebugTool.

2. Comm Param (communication parameters): Enter values according to the communication interfaces and frequencies supported by the Host. Multiple communication methods can be evaluated simultaneously.

3. Host Real-Time Param (Host real-time parameters): If exact data is unavailable, enter values based on Appendix A, or measure them using the port_device_time_meas project (for details, see Section 4.2.1, SPI Transmission Performance Test).

4. Other Param (other parameters):

   + Other Time: A reserved-time input. For example, reserve 5 ms per frame for processing high-priority tasks.

   + Trans data length (length of data to be transmitted): Length of the data to be transmitted.

Output data:

1. Trans Time (transmission time): Evaluates whether sufficient time is available within the frame period to complete data transmission.

   A positive margin indicates that sufficient time is available. A larger value indicates more margin and may allow the transmission rate to be reduced.

   A negative margin indicates that the transmission time exceeds the frame period. Increase the transmission rate and check whether the margin becomes positive; otherwise, the communication method cannot be used.

2. Device Idle Time (radar Device idle time): Frame period minus the hardware transmit/receive interval and data-transmission time. For low-power applications, this value is used to evaluate the available sleep time.

3. Max trans data: Evaluates the radar-data volume and parameters that can be supported at the highest transmission bandwidth supported by the Host.

3.2.2 Memory Resource Evaluation

The Memory sheet in Radar_HIF_Calculation provides a table for quickly obtaining the amount of Flash (static storage or XIP) and SRAM required by HIF.

3.2.2.1 Device Memory

As shown below, configure the Input options to obtain the final Total memory size.

1. The DLL item under Base is mandatory. Select one PHY communication method. This communication method can be determined using the parameter-design section below.

2. Extent contains extended functions. Fragmentation (Split) is enabled by default. For application-layer and transport-layer retransmission, see the HIF Development Manual. These functions are disabled by default.

3. App indicates the additional transmission buffer required when an application is enabled.

(1) Data Cube Size indicates the amount of data to be transmitted and can be 1/2/4/8/16/32/64/128/256/512 KB. Obtain this value from the Data Cube evaluation sheet or calculate it according to the *'RS6x_7x_mmWave_Radar_Evaluation_Quick_Start_Guide'*. Report Period indicates the reporting period. A value of 1 means that data is reported promptly for each frame; a value of 2 means that two frames can be buffered and reported together, and so on. This value is related to the Host response latency and frame period (see Section 3.2.1.1) and corresponds to Device Cached frames in the timing evaluation.

(2) Point Cloud indicates the number of each type of point-cloud point to be transmitted (see Section 3.2.1.1).

Device Memory Size					Flash (KB)	SRAM (KB)
Base	DLL	Command/Response length(Bytes)		512	10.64	11.61
		HIF thread stack(Bytes)		1024		
		Report data node counter		64		
		Report frame node counter		16		
		Block semaphore counter		4		
	PHY	SPI		Y	0.34	0.44
		UART		N	0	0
		I2C		N	0	0
Exten	Split	Split node counter	4	Y	3.03	1.45
	Retry	Application layer retry		N	0	0
		Transfer layer retry		N	0	0
	PM	Power Manage		N	0	0
App	DataCube	Data Cube Size(KB)	32	N	0	0.00
		Device cache frames	1			
	PointCloud	motion	768	N	0	0.00
		presence	256			
Total					14.01	13.50

Input

Output

Figure 3-4 Device Memory

3.2.2.2 Host Memory

Host Driver Flash and SRAM sizes differ only slightly between specifications and are treated as the base size. Application SRAM usage is the primary evaluation item and is related to the application-data size and number of buffered frames.

1. For Data Cube application data, if Cube Size is 64 KB and two frames are buffered, the required application SRAM is 64 KB × 2 = 128 KB.

2. For point-cloud application data, if motion cloud is 768, presence cloud is 256, and two frames are buffered, the required SRAM buffer is (768 + 256) × 12 × 2 B = 24 KB.

Note: Host Cache frames must correspond to Host Cache frames in the timing evaluation. In the timing evaluation, this parameter affects the solution for non-uniform Host computing availability.

Host Memory Size					Flash (KB)	SRAM (KB)
Driver	HIF host driver				17	4.5
App	DataCube	Data Cube Size(KB)	32	N	0	0.00
		Cached frames	1			
	PointCloud	motion	768	N	0	0.00
		presence	256			
		Host Cached frames	2			
Total					17.00	4.50

Input

Output

Figure 3-5 Host Memory

3.2.3 I/O Resources

The I/O sheet in *'Radar_HIF_Calculation'* provides an I/O configuration table, as shown below:

1. Select only the communication method and print output.

(1) Determine the communication method based on Section 3.2.1, Timing Evaluation.

(2) The print output can be used for Device debugging and can be implemented as a test point without connection to the Host.

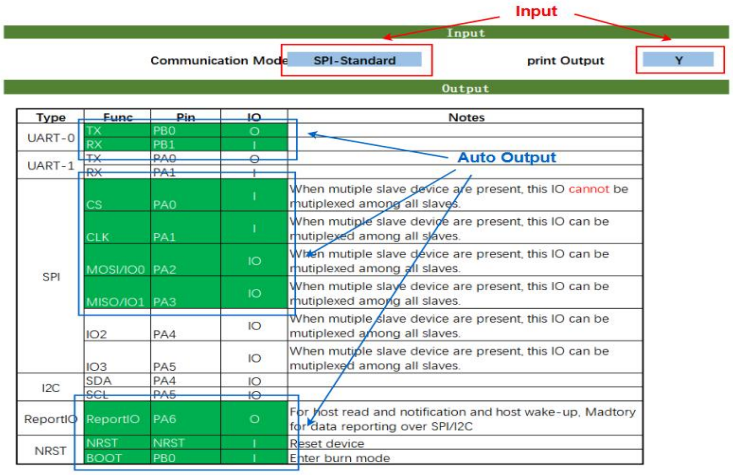
2. Automatically matched I/Os are shown in green.

(1) This table is shown from the Device perspective. The Host must provide matching functions.

(2) The user must connect the BOOT pin for firmware burn.

(3) NRST is used to enter burn mode and reset the Device. If the module has an independent power supply, these operations can also be implemented by cycling the power.

For multiple slaves, note which I/Os can be shared. For I^2C communication, SDA and SCL can be shared. For SPI communication, CS cannot be shared, while the other I/Os can be shared. For UART communication, Tx and Rx cannot be shared. For SPI and I^2C communication, Report I/O cannot be shared.



Type	Func	Pin	IO	Notes
UART-0	TX	PB0	O	
	RX	PB1	I	
UART-1	TX	PA0	O	
	RX	PA1	I	
SPI	CS	PA0	I	When mutiple slave device are present, this IO cannot be mutiplexed among all slaves.
	CLK	PA1	I	When mutiple slave device are present, this IO can be mutiplexed among all slaves.
	MOSI/IO0	PA2	IO	When mutiple slave device are present, this IO can be mutiplexed among all slaves.
	MISO/IO1	PA3	IO	When mutiple slave device are present, this IO can be mutiplexed among all slaves.
	IO2	PA4	IO	When mutiple slave device are present, this IO can be mutiplexed among all slaves.
	IO3	PA5	IO	When mutiple slave device are present, this IO can be mutiplexed among all slaves.
I2C	SDA	PA4	IO	
	SCL	PA5	IO	
ReportIO	ReportIO	PA6	O	For host read and notification and host wake-up, Madtory for data reporting over SPI/I2C
NRST	NRST	NRST	I	Reset device
	BOOT	PB0	I	Enter burn mode

Figure 3-6 I/O Resources

4 Development and Debugging

After determining the parameters, begin HIF communication debugging according to the following procedure:

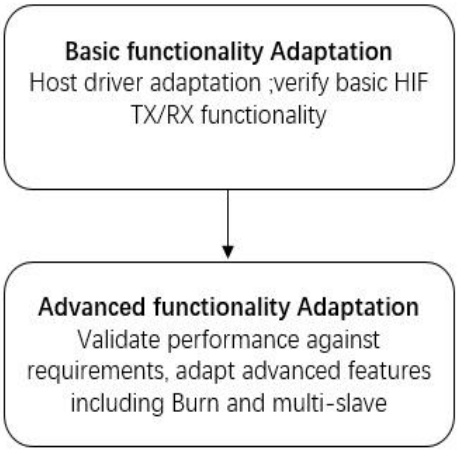


Figure 4-1 Development and Debugging Procedure

4.1 Basic-Function Adaptation

The purpose of function adaptation is to port the Host Driver and establish communication with the Device. Follow the procedure below:

1. Port the Host Driver to the target platform and compile it successfully.
2. Burn hif_port_device_xxx.img (where xxx indicates the communication method and parameters, such as uart_921600) to the Device.
3. Debug communication between the Host and Device.

4.1.1 Host Porting

Perform adaptation and porting according to Section 5.1, Host Driver Porting, in the *'RS6x_7x_HIF_Reference_Manual'*.

4.1.2 Porting and Debugging

After adapting the Host Driver, use hif_port_host_sample on the Host and hif_port_device_sample on the Device for communication debugging.

Note:

Use the hif_port_device_sample image provided in the product package directly. For example:

1. hif_port_device_uart_921600.img: UART communication at 921600 bps
2. hif_port_device_i2c_3a.img: I^2C communication, software-compatible with 100 kHz/400 kHz, communication address 0x3A (7 bits)
3. hif_port_device_spi_std_64mhz.img: Single-line SPI communication, supporting a maximum communication frequency of 64 MHz
4. hif_port_device_spi_dual_64mhz.img: Dual-line SPI communication, supporting a maximum communication frequency of 64 MHz
5. hif_port_device_spi_quad_64mhz.img: Quad-line SPI communication, supporting a maximum communication frequency of 64 MHz

The flow for this pair of Samples is as follows:

1. At 1-second intervals, the Device reports an incrementing sequence number.
2. The Host then sends a Command to obtain the version number and receives the returned version information.

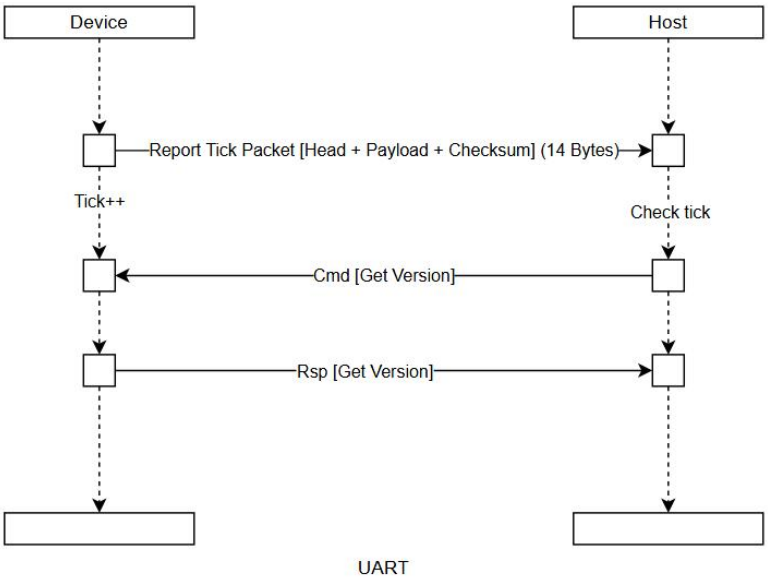


Figure 4-2 Port Sample UART Transmission Flow

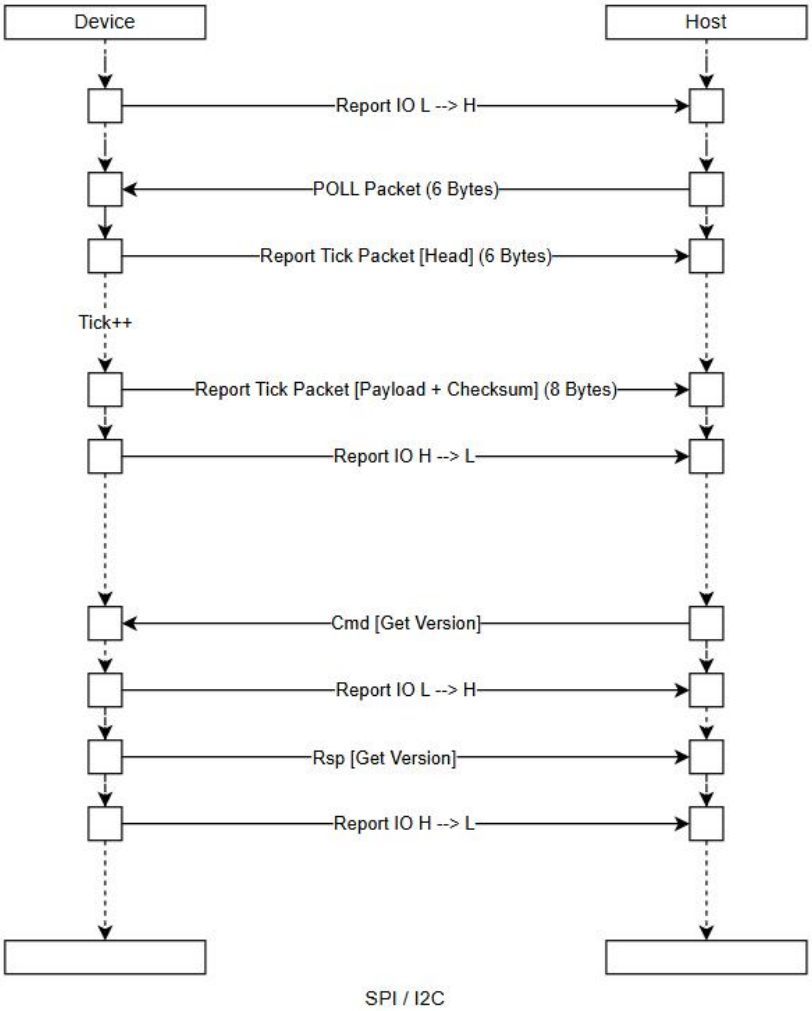


Figure 4-3 Port Sample SPI or I^2C Transmission Flow

For UART, debug the data-reception path first, followed by the data-transmission path. For SPI/I^2C, debug the I/O notification flow first, followed by the data-transmission path and finally the data-reception path.

4.1.2.1 I/O Notification Flow

1. First, verify that port_io receives the notification. By default, a GPIO rising-edge interrupt is used as the trigger.
2. If the port receives the I/O trigger, verify that llc_notify_io_irq_handle receives the trigger.
3. Finally, verify that HIF_TL_DataNotify_CB receives the trigger. The I/O notification flow is then operational.

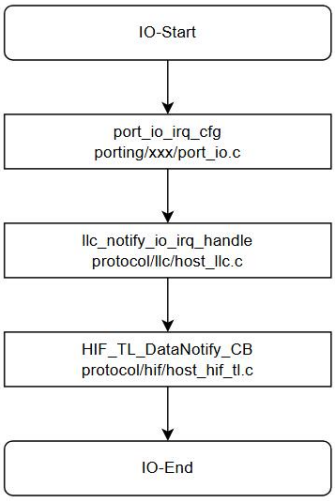


Figure 4-4 I/O Notification Flow

4.1.2.2 Data Transmission Flow

1. First, verify that the transport layer calls HIF_TL_SendMsg to send the Get_Version Command or calls HIF_TL_SendPoll to send a Poll for SPI/I^2C transmission.
2. Verify that LLC_Send is executed at the link layer.
3. Verify that the port layer is called. If the Device has not received the data, verify that the port-driver configuration is correct, for example, whether pin multiplexing is configured correctly and whether the clock is enabled. A logic analyzer can be used to capture port signals and confirm the transmitted data.

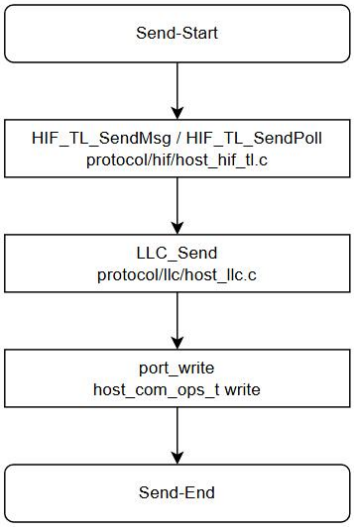


Figure 4-5 Data Transmission Notification Flow

4.1.2.3 Data Reception Flow

1. First, verify that the port read interface receives the data correctly. If not, check the driver configuration. A logic analyzer can be used to capture and analyze the port signals.

2. Then verify that the link layer and transport layer receive the data correctly.

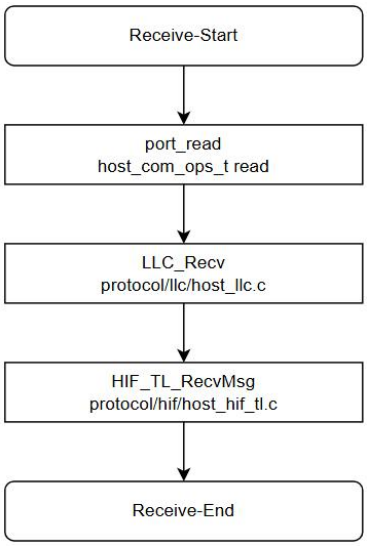


Figure 4-6 Data Reception Notification Flow

4.2 Advanced-Function Adaptation

After basic communication is operational as described in the preceding section, continue debugging the following additional basic functions as required:

1. SPI transmission performance and Host real-time evaluation

   For solutions such as Data Cube transmission that require high transmission bandwidth and use a Linux-class Host, this Sample can be used to measure Host real-time parameters, such as response latency, to accurately evaluate whether the bandwidth is sufficient.

2. Image Burn Procedure

   The Host must update the Device firmware by directly writing the image to Device Flash.

3. Multiple-slave debugging

   If the Host must control multiple Devices simultaneously, use this Sample to verify the function.

4. Sample debugging

   The functional verification of the basic Samples (Table 2-1, Basic Samples) can be used as a foundation and reference for user applications.

4.2.1 SPI Transmission Performance Test

Burn hif_port_device_time_meas.img to the Device, run hif_port_host_sample on the Host, and observe the Host output.

At 1-second intervals, the Device reports a 32 KB Data Cube. The Device measures the Host transmission time to calculate the SPI transmission frequency, Host response latency, and other parameters. After collecting statistics for a period, the Device reports the average and maximum values to the Host, and the Host prints the results.

Note: The results are for reference only. A logic analyzer provides more accurate measurements.

```
          type:        avg        max
report-latency:        131        146
     poll-delay:        553        554
     head-delay:         86         87
  inter-packet:          94         94
      spi-clock:         16         16
```

Figure 4-7 Host Real-Time Parameters

In the preceding figure, avg indicates the statistical average and max indicates the statistical maximum. Report-latency indicates the Host response latency. The other parameters correspond to Appendix B: Host Real-Time Parameters.

4.2.2 Burn Debugging

Use burn_sample on the Host to verify the burn function. Adapt the Device UPD and RST pins and the image-access interface in port_store.c.

The main flow is as follows:

```
for (int count = 0; count < HIF_BURN_RETRY_COUNT; count++) {
    do {
        /* host enter burn mode */
        status = Host_BurnMode_Enter(devHandle);
        if (status != HOST_ERRCODE_SUCCESS) {
            HIF_LOG_PRINT("device(%p) enter burn mode fail(%d)\n", devHandle, status);
            break;
        }

        /* host burn image */
        status = Host_Burn_Image(devHandle, "image_user");
        if (status != HOST_ERRCODE_SUCCESS) {
            HIF_LOG_PRINT("device(%p) burn image fail(%d)\n", devHandle, status);
        }

    } while(0);

    HIF_LOG_PRINT("exit burn mode\n");
    /* host exit burn mode */
    Host_BurnMode_Exit(devHandle);
    if (status == HOST_ERRCODE_SUCCESS) {
        break;
    }
}
```

4.2.3 Multiple-Slave Debugging

Use multi_device_sample for multiple slaves. The Sample can respond to reports from different Devices.

4.2.4 Sample Debugging

Run cmd_and_report_sample on the Host together with the five basic Device Samples to verify data reporting and Command transmission.

5 Appendix A: HIF Radar Reporting Timing

This chapter describes radar-data generation and processing and the HIF reporting flow. As shown below, the Device operates on a frame basis, and each frame consists of three parts: hardware transmission/reception, data processing, and the Idle state. HIF communication between the Device and Host and Host data processing both use this timeline as a reference.

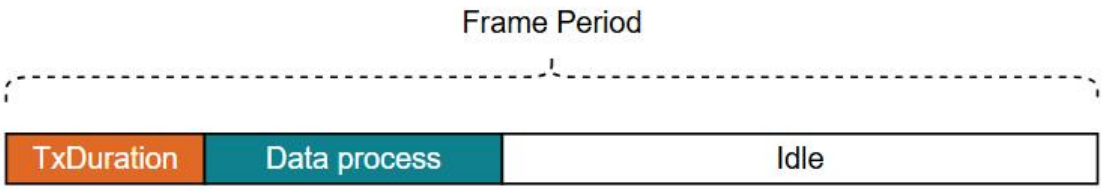


Figure 5-1 Radar Frame Period

5.1 Point-Cloud Data Reporting

During the Data processing phase, the Device processes the raw Data Cube data into a point cloud and reports it to the Host through HIF. Reporting can use either of the following methods:

1. Single-frame processing: The radar frame period is relatively long and the Host has strong real-time performance, allowing the current-frame data to be transmitted and processed within each frame.
2. Multi-frame processing: The radar frame period is relatively short and the reported data volume is small, allowing data from multiple frames to be accumulated for reporting and processing.

5.1.1 Single-Frame Processing

With this method, the Device and Host each require only one point-cloud buffer, minimizing SRAM usage. However, the current-frame data must be processed before the next frame is uploaded. As shown below, the maximum effective interval for Device data upload (Time for transmitting data) is one frame period. If the Host is a platform with high response latency, such as Linux, the effective transmission time is Time_for_transmitting_data - Host_response_delay. On the Host, when multiple buffers are used, the CPU computing capability available for data processing during HIF reception must be derated according to whether DMA or multi-core processing is used.

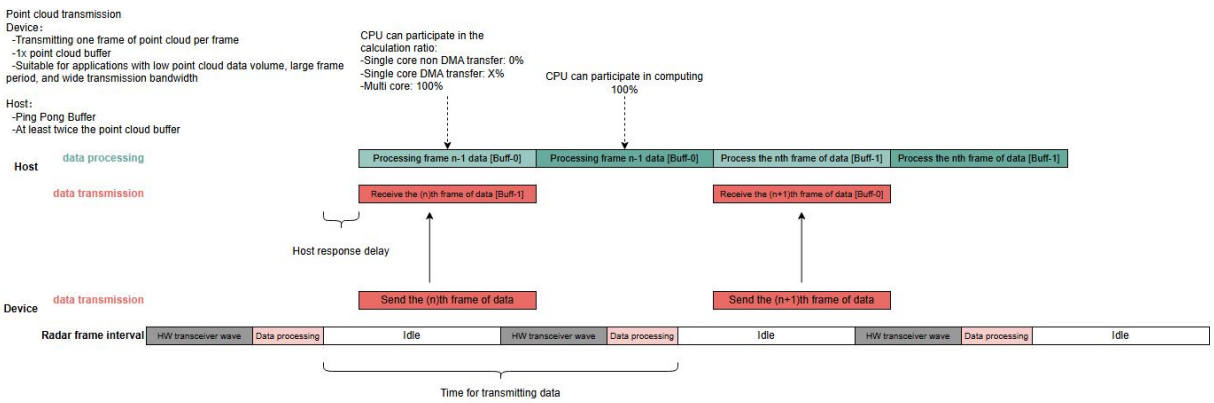


Figure 5-2 Single-Frame Point-Cloud Reporting

5.1.2 Multi-Frame Processing

With multi-frame reporting, multiple buffers on the Device relax the Host real-time requirements, while multiple buffers on the Host provide more processing time. This method is recommended for applications with a short frame period and a small data volume, for example, a 5-ms or 10-ms frame period and fewer than 128 point-cloud points.

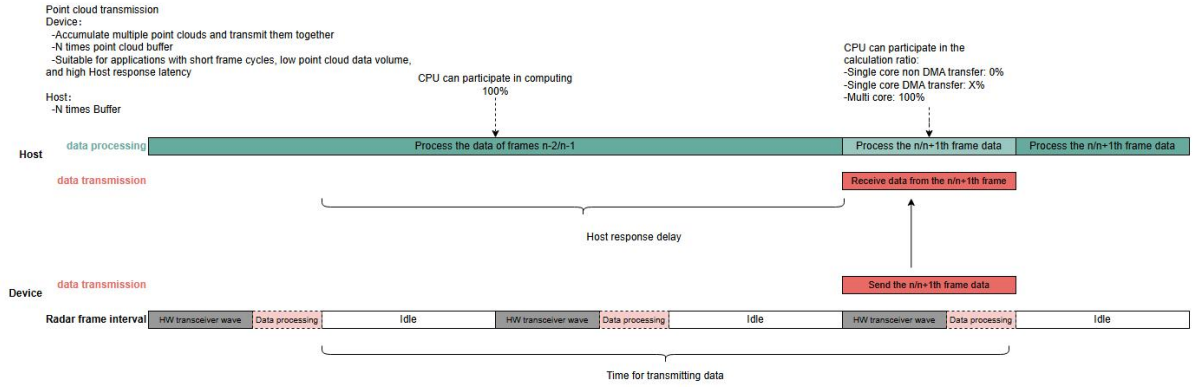


Figure 5-3 Multi-Frame Point-Cloud Reporting

5.2 Data Cube Reporting

During the HW transceiver wave phase, the software cannot access the Data Cube buffer because it must be used exclusively by the hardware. The Data processing phase performs no, or very little, data processing. During the Idle phase, data can be copied to the HIF buffer and reported to the Host. Reporting can use one of the following three methods:

1. For an application with a large reported-data volume and a long radar frame period, the current-frame data can be transmitted and processed within each frame.
2. For an application with a large data volume and a short radar frame period, transmission is completed during the Idle interval and the Host can buffer multiple frames for processing.
3. For an application with a small data volume and a short radar frame period, or a scenario with high Host response latency, the Device can buffer multiple frames and report them together.

5.2.1 Single-Frame Reporting and Processing

The time available for HIF data transmission is only Idle - Host_response_delay. Host data processing cannot extend into the next frame. This method is suitable for applications with a long frame period and low Host response latency. It requires the fewest buffers on the Host and Device: the Host requires only one buffer, and the Device can use a smaller sliding buffer.

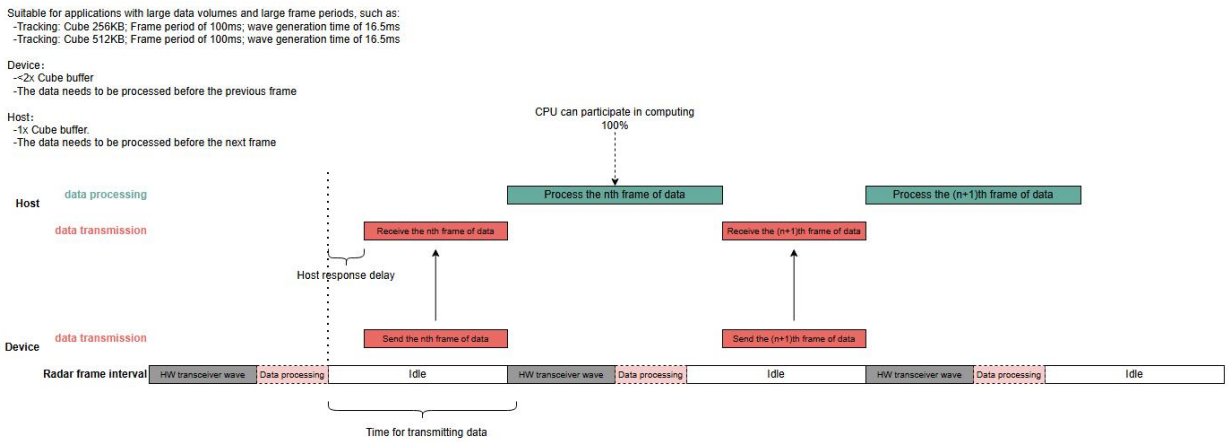


Figure 5-4 Single-Frame Data Cube Reporting and Processing

5.2.2 Single-Frame Reporting and Multi-Frame Processing

Data is transmitted as described in Single-Frame Reporting and Processing. The Host can buffer multiple frames for processing to address non-uniform Host computing availability, where high computing capability cannot be provided continuously because the task may be preempted by a higher-priority task.

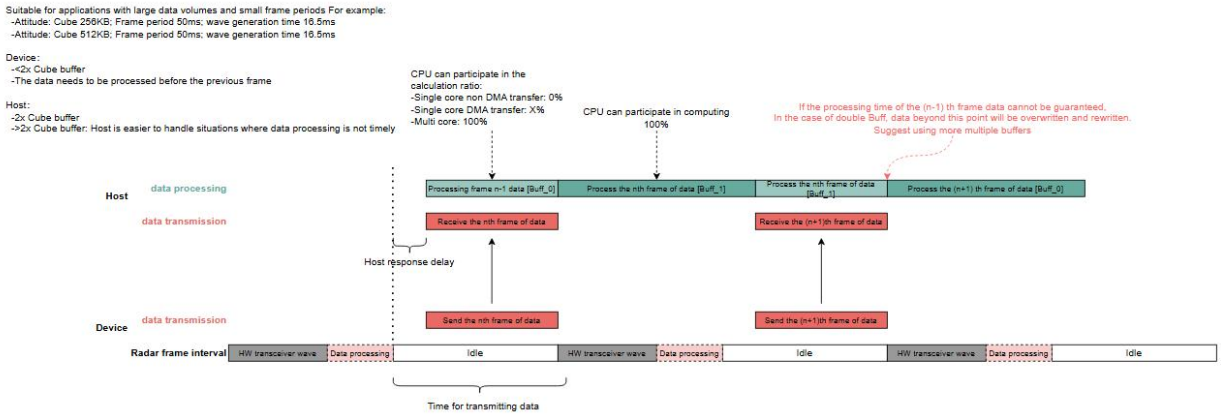


Figure 5-5 Single-Frame Data Cube Reporting and Multi-Frame Processing

5.2.3 Multi-Frame Reporting and Processing

For applications with high Host response latency, such as Linux, or with a short frame period and small data volume, data from multiple frames can be accumulated for reporting. The Device and Host require more buffers, but transmission can proceed in parallel with HW transceiver wave and Data processing.

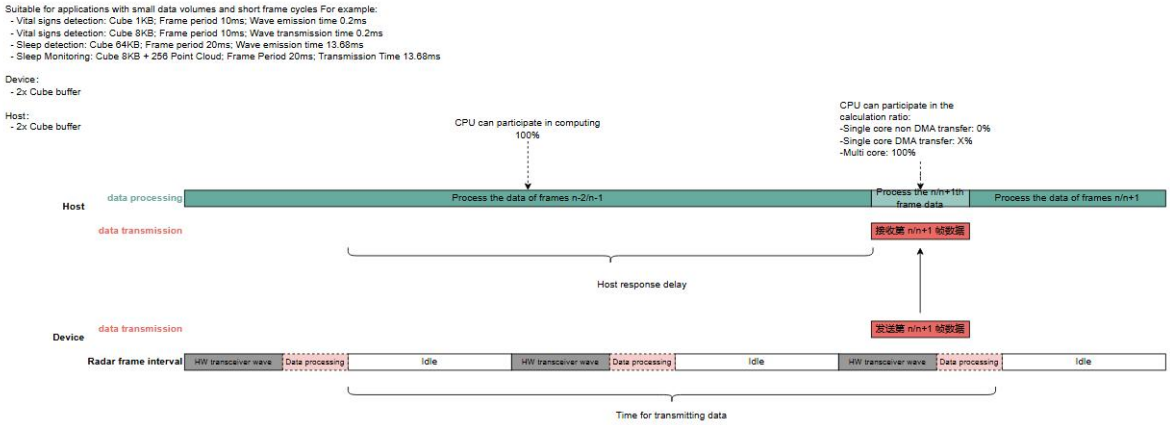


Figure 5-6 Multi-Frame Data Cube Reporting and Processing

5.2.4 Other Information Reporting

Other information, such as user-processing results or debug information, is typically reported independently of the frame period and can be reported at any time. The timing diagram is similar to Multi-Frame Point-Cloud Processing.

6 Appendix B: Host Real-Time Parameters

In SPI/I^{2}C transmission mode, the following four Host real-time parameters affect transmission efficiency.

Table 6-1 Host Real-Time Parameters

Item	Notes	RTOS	Linux
Response Latency	Delay from assertion of Report I/O until the Host sends the Poll packet. For Data Cube transmission, this delay affects the first packet; subsequent cascaded packets are not affected. The value depends on the Host I/O response performance.	50 μs~2 ms	10 ms ~ 20 ms
poll-delay	Time from when the Host sends the Poll packet until the Device prepares the data packet and the Host reads the packet Header. The value depends on Device Poll-processing performance and Host read-switching performance.	500 μs	1 ms
Head-delay	Delay from when the Host reads the Header of the reported Device data until it reads the Payload. The value depends on Host continuous-reading performance and Header Checksum calculation performance.	50 μs	50 μs
Inter-packet	Delay from when the Host finishes reading the reported data until it reads the Header of the next packet. The value depends on Host continuous-reading performance and Payload Checksum calculation performance.	100 μs	50 μs

The corresponding timing diagram is shown below:

Figure 6-1 Host Real-Time Parameters

Disclaimer and Copyright Notice

The information provided by our company in this document is accurate to the best of our knowledge, but we do not guarantee that the content is completely error-free. All liability for any damages arising from the use of this document (including but not limited to indirect, incidental, or special losses) or any improper use resulting from non-compliance with the instructions herein (including but not limited to over-voltage, over-frequency, or over-temperature operations) is explicitly disclaimed.

The content in this document, including URL references, may differ from the actual content due to product version upgrades or other reasons, and is subject to change without notice.

THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES. NO WARRANTY IS PROVIDED TO THIS DOCUMENT FOR ITS MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, NOR DOES ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

All liability, including liability for infringement of any patent rights, relating to use of information in this document is disclaimed. No licenses express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

Copyright © 2026 Possumic Technology Co., Ltd. All rights reserved.